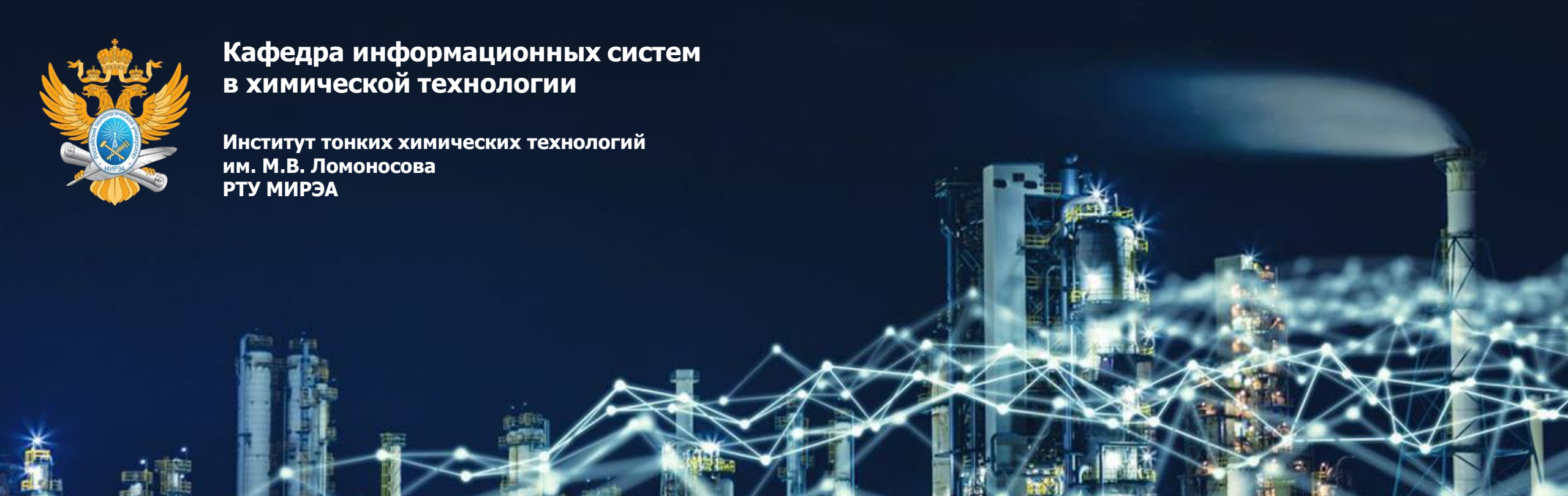




**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХИМИЧЕСКОЙ ТЕХНОЛОГИИ



Лекция 7. Файловые системы и работа с файлами

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

- 🕒 Управление хранением
- 🕒 Файловые системы и точки монтирование
 - Файловые системы, устройства хранения и блочные устройства
 - Разделы дисков и логические тома
 - Монтирование и размонтирование файловых систем
- 🕒 Управление файлами
 - Иерархия файловой системы Linux
 - Имена файлов и пути к ним
 - Управление файлами и директориями
 - Создание ссылок между файлами
 - Использование шаблонов и переменных в командной строке
- 🕒 Поиск файлов в файловых системах

Управление хранением

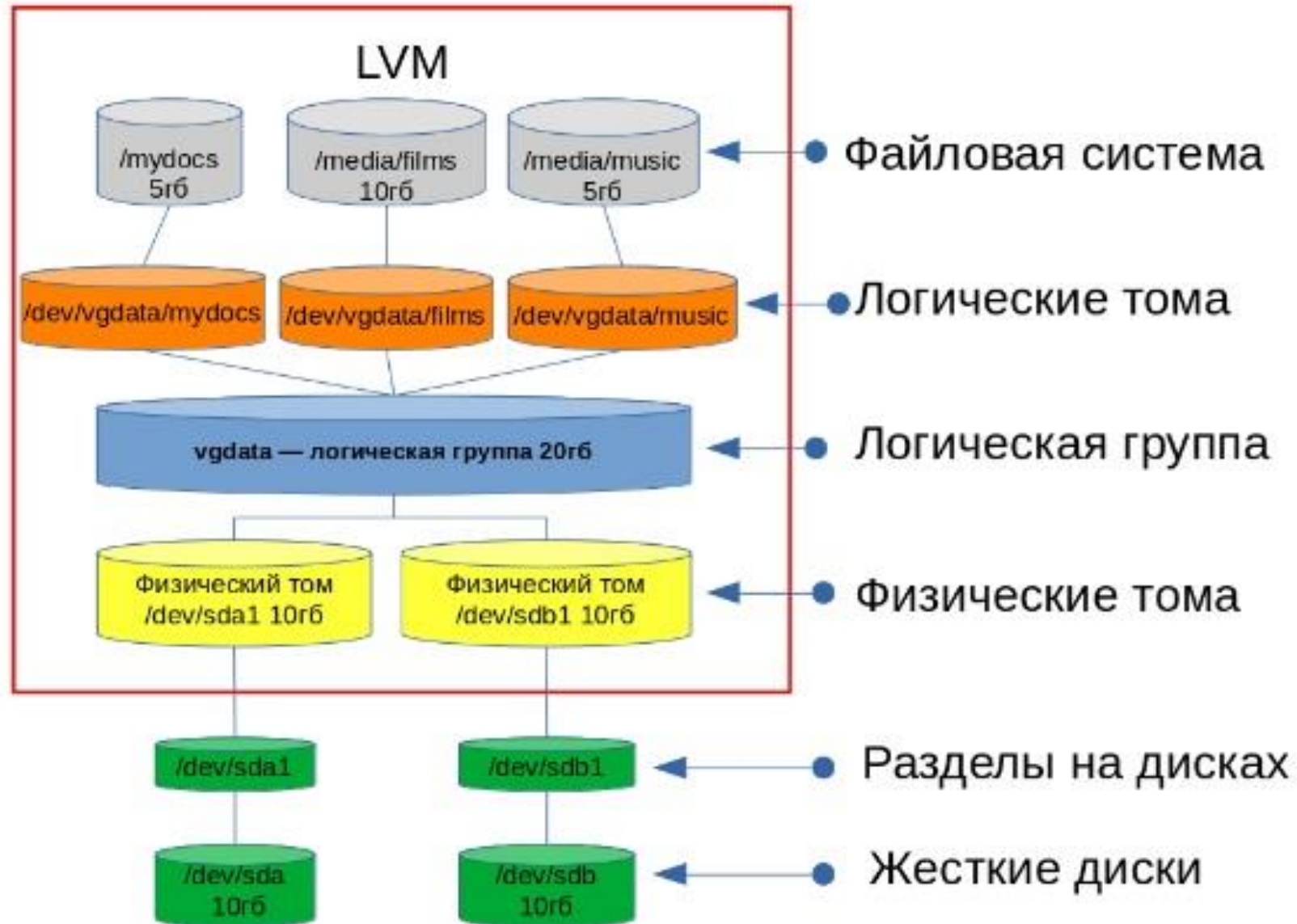
Файловые системы и точки монтирование

- Файловые системы, устройства хранения и блочные устройства
- Разделы дисков и логические тома
- Монтирование и размонтирование файловых систем

Управление файлами

- Иерархия файловой системы Linux
- Имена файлов и пути к ним
- Управление файлами и директориями
- Создание ссылок между файлами
- Использование шаблонов и переменных в командной строке

Поиск файлов в файловых системах



Следующая тема

- 🕒 Управление хранением
- 🕒 **Файловые системы и точки монтирование**
 - **Файловые системы, устройства хранения и блочные устройства**
 - Разделы дисков и логические тома
 - Монтирование и размонтирование файловых систем
- 🕒 **Управление файлами**
 - Иерархия файловой системы Linux
 - Имена файлов и пути к ним
 - Управление файлами и директориями
 - Создание ссылок между файлами
 - Использование шаблонов и переменных в командной строке
- 🕒 Поиск файлов в файловых системах

Файловые системы, хранилища и блочные устройства.

Низкоуровневый доступ к устройствам хранения в Linux обеспечивается файлом особого типа, называемым блочным устройством (**block device**).

Присвоение имен блочным устройствам:

ТИП УСТРОЙСТВА	ШАБЛОН НАЗВАНИЯ УСТРОЙСТВА
SATA/SAS/USB -накопитель	<code>/dev/sda, /dev/sdb ...</code>
Паравиртуализированное хранилище virtio-blk (некоторые виртуальные машины)	<code>/dev/vda, /dev/vdb ...</code>
Хранилище, подключенное к NVMe (многие твердотельные накопители SSD)	<code>/dev/nvme0, /dev/nvme1 ...</code>
SD/MMC/eMMC хранилище (SD -карты)	<code>/dev/mmcblk0, /dev/mmcblk1 ...</code>

Файловые системы и точки монтирования.

Изучение файловых систем.

Команда **df**

В следующем примере показаны файловые системы и точки монтирования на хосте:

```
user@host:~$ df
Filesystem      1K-blocks    Used   Available   Use%    Mounted on
Devtmpfs         912584        0    912584        0%      /dev
Tmpfs            936516        0    936516        0%      /dev/shm
Tmpfs            936516   16812    919704        2%      /run
Tmpfs            936516        0    936516        0%      /sys/fs/cgroup
/dev/vda3       8377344 1411332    6966012       17%      /
/dev/vda1       1038336  169896    868440       17%      /boot
Tmpfs           187300        0    187300        0%      /run/user/1000
```

Изучение файловых систем.

Команда **df**

В следующем примере показан отчет о файловых системах в хост-системе со всеми модулями, преобразованными в удобочитаемый формат:

```
user@host:~$ df -h
Filesystem      1K-blocks    Used   Available   Use%    Mounted on
Devtmpfs         892M         0        892M        0%      /dev
Tmpfs            915M         0        915M        0%      /dev/shm
Tmpfs            915M        17M        899M        2%      /run
Tmpfs            915M         0        915M        0%      /sys/fs/cgroup
/dev/vda3         8.0G        1.4G        6.7G       17%      /
/dev/vda1        1014M        166M        849M       17%      /boot
Tmpfs            183M         0        183M        0%      /run/user/1000
```


Файловые системы и точки монтирования.

Изучение файловых систем.

Команда **du**

В следующем примере показан отчёт об использовании диска для каталога **/usr/share** на хосте:

```
root@host:~# du /usr/share
...output omitted...
176 /usr/share/smartmontools
184 /usr/share/nano
8 /usr/share/cmake/bash-completion
8 /usr/share/cmake
356676 /usr/share
```

Показать отчёт об использовании диска в удобочитаемом формате для каталога **/usr/share** на хосте:

```
root@host:~# du -h /var/log
...output omitted...
176K /usr/share/smartmontools 184K /usr/share/nano
8.0K /usr/share/cmake/bash-completion
8.0K /usr/share/cmake 369M /usr/share
```

Следующая тема

- 🔄 Управление хранением
- 🔄 **Файловые системы и точки монтирование**
 - Файловые системы, устройства хранения и блочные устройства
 - **Разделы дисков и логические тома**
 - Монтирование и размонтирование файловых систем
- 🔄 Управление файлами
 - Иерархия файловой системы Linux
 - Имена файлов и пути к ним
 - Управление файлами и директориями
 - Создание ссылок между файлами
 - Использование шаблонов и переменных в командной строке
- 🔄 Поиск файлов в файловых системах

Разделы дисков и логические тома.

Разделы диска.

Длинный список файла устройства **/dev/sda1** на хосте показывает его специальный тип файла как **b**, что означает блочное устройство:

```
user@host:~$ ls -l /dev/sda1  
brw-rw----. 1 root disk 8, 1 Feb 22 08:00 /dev/sda1
```

Логические тома.

Другой способ организации дисков и разделов - управление логическими томами (**logical volume management LVM**). С помощью **LVM** одно или несколько блочных устройств могут быть объединены в пул хранения, называемый группой томов (**volume group**). Затем дисковое пространство в группе томов распределяется по одному или нескольким логическим томам (**logical volumes**), которые являются функциональным эквивалентом раздела, находящегося на физическом диске. Система **LVM** присваивает имена группам томов и логическим томам при создании. **LVM** создает каталог в **/dev**, который соответствует имени группы, а затем создает символическую ссылку в этом новом каталоге с тем же именем, что и логический том. После этого файл логического тома можно будет смонтировать. Например, если группа томов называется **myvg**, а логический том в ней называется **mylv**, то полный путь к файлу устройства логического тома будет **/dev/myvg/mylv**.

Следующая тема

- 🔄 Управление хранением
- 🔄 **Файловые системы и точки монтирование**
 - Файловые системы, устройства хранения и блочные устройства
 - Разделы дисков и логические тома
 - **Монтирование и размонтирование файловых систем**
- 🔄 Управление файлами
 - Иерархия файловой системы Linux
 - Имена файлов и пути к ним
 - Управление файлами и директориями
 - Создание ссылок между файлами
 - Использование шаблонов и переменных в командной строке
- 🔄 Поиск файлов в файловых системах

Монтирование и размонтирование файловых систем.

🔄 Установка файловых систем вручную

- Есть два распространенных способа указать файловую систему на разделе диска для команды **mount**:
 - С именем файла устройства в `/dev`, содержащего файловую систему.
 - С UUID, записанным в файловую систему, универсальный уникальный идентификатор.

🔄 Идентификация блочного устройства:

Используйте команду **lsblk**, чтобы вывести список деталей указанного блочного устройства или всех доступных устройств.

```
root@host:~# lsblk
NAME                MAJ:MIN RM  SIZE RO  TYPE MOUNTPOINT
vda                  253:0    0   12G  0   disk
├─vda1               253:1    0    1G  0   part  /boot
├─vda2               253:2    0    1G  0   part  [SWAP]
└─vda3               253:3    0   11G  0   part  /
vdb                  253:16    0   64G  0   disk
└─vdb1               253:17    0   64G  0   part
```

Монтирование и размонтирование файловых систем.

🔄 Монтирование по имени блочного устройства

В следующем примере файловая система монтируется в разделе **/dev/vdb1** в каталоге **/mnt/data**:

```
root@host:~# mount /dev/vdb1 /mnt/data
```

🔄 Монтирование по UUID файловой системы

Команда **lsblk -fp**

```
root@host:~# lsblk -fp
NAME                                FSTYPE LABEL  UUID                                  MOUNTPOINT
/dev/vda
├─/dev/vda1  xfs                23ea8803-a396-494a-8e95-1538a53b821c /boot
├─/dev/vda2  swap              cdf61ded-534c-4bd6-b458-cab18b1a72ea [SWAP]
└─/dev/vda3  xfs                44330f15-2f9d-4745-ae2e-20844f22762d /
/dev/vdb
└─/dev/vdb1  xfs                46f543fd-78c9-4526-a857-244811be2d88
```

Монтирование файловой системы по **UUID**.

```
root@host:~# mount UUID="46f543fd-78c9-4526-a857-244811be2d88" /mnt/data
```


Монтирование и размонтирование файловых систем.

Размонтирование файловых систем.

Чтобы размонтировать файловую систему, команда **umount** ожидает точку монтирования в качестве аргумента.

```
root@host:~# umount/mnt/data
```

В приведённом ниже примере происходит сбой выполнения команды **umount** из-за того, что файловая система используется (оболочка использует **/mnt/data** в качестве текущего рабочего каталога), генерируя сообщение об ошибке.

```
root@host:~# cd /mnt/data
root@host:/data# umount /mnt/data
umount: /mnt/data: target is busy
```

Команда **lsuf** выводит список всех открытых файлов и процессов, обращающихся к ним, в указанном каталоге. Полезно определить, какие процессы в настоящее время препятствуют успешному размонтированию файловой системы.

```
root@host:~# lsuf /mnt/data
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
bash	1593	root	cwd	DIR	253,17	6	128	/mnt/data
lsuf	2532	root	cwd	DIR	253,17	19	128	/mnt/data
lsuf	2533	root	cwd	DIR	253,17	19	128	/mnt/data

Следующая тема

- 🔄 Управление хранением
- 🔄 Файловые системы и точки монтирование
 - Файловые системы, устройства хранения и блочные устройства
 - Разделы дисков и логические тома
 - Монтирование и размонтирование файловых систем
- 🔄 **Управление файлами**
 - Иерархия файловой системы Linux
 - Имена файлов и пути к ним
 - Управление файлами и директориями
 - Создание ссылок между файлами
 - Использование шаблонов и переменных в командной строке
- 🔄 Поиск файлов в файловых системах

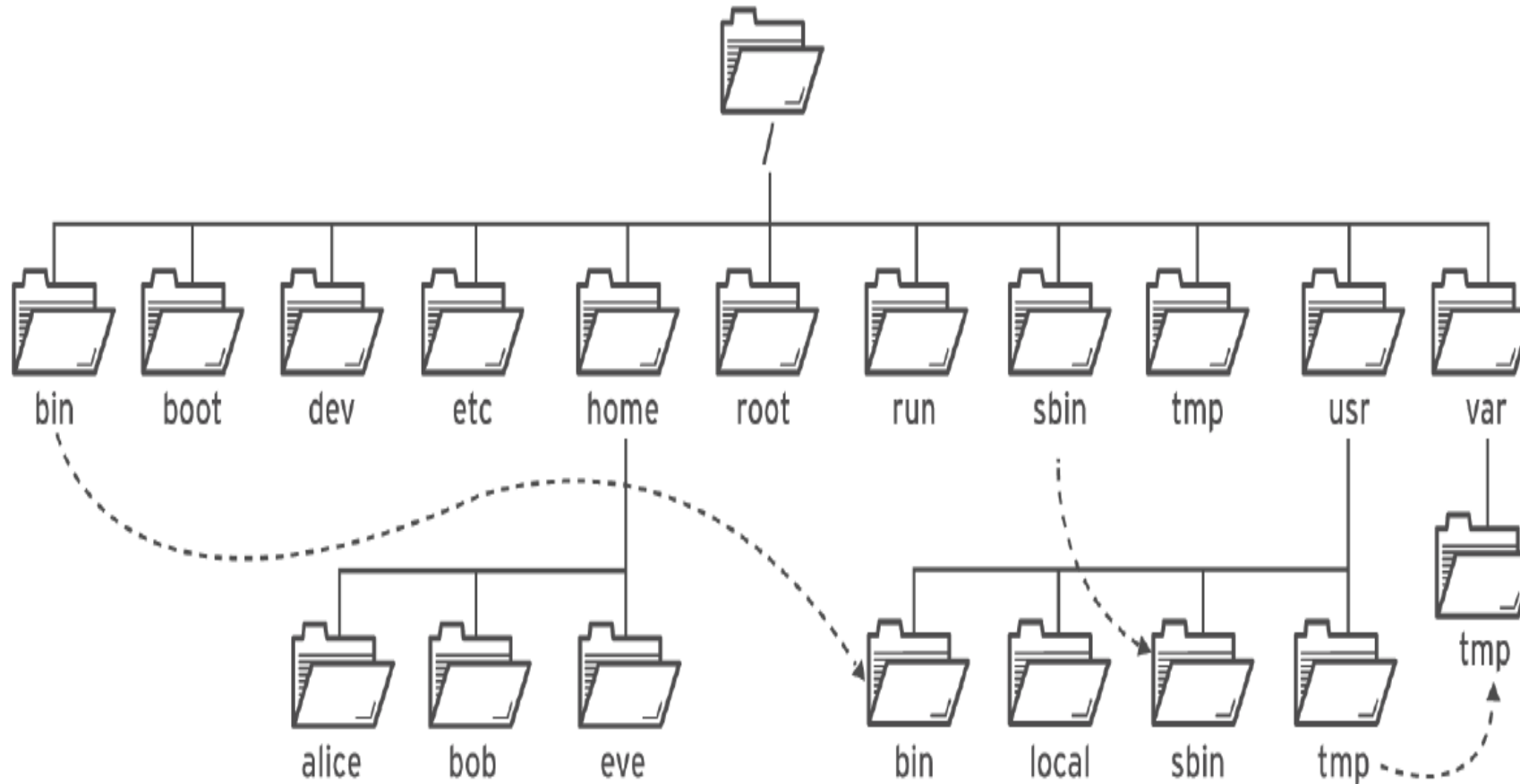
Некоторые особенности

- Astra Linux в качестве файловых систем на носителях информации компьютеров с ОС (в том числе на съемных машинных носителях информации) должны использоваться только файловые системы ext2/ext3/ext4, обеспечивающие гарантированное уничтожение (стирание) информации.
- Особенности Ext4
 - Совместимость (Обновление до ext4 без переформатирования и без переустановки ОС и программ)
 - Большой размер файлов и файловой системы (48-битная адресация блоков и размер файла до 16 терабайт)
 - Масштабируемость подкаталогов (т.е. без ограничений по количеству)
 - Экстенты (Способствуют непрерывному размещению данных на носителе)
 - Контрольные суммы журнала (улучшаются одновременно и надёжность, и производительность)

Иерархия файловой системы

🔄 Дерево каталогов, известное как иерархия файловой системы.

/ обозначает корневой каталог



Иерархия файловой системы

МЕСТО РАСПОЛОЖЕНИЯ	ЦЕЛЬ
/usr	/usr/bin : пользовательские команды. /usr/sbin : команды системного администрирования /usr/local : локальное программное обеспечение.
/etc	Файлы конфигурации, относящиеся к этой системе.
/var	Файлы, которые динамически изменяются, такие как базы данных, каталоги кэша, файлы журналов, буферные документы принтера и содержимое веб-сайтов
/run	Файлы текущего состояния приложений, запущенных с момента последней загрузки.
/home	Личные данные и файлы конфигурации пользователей
/root	Домашний каталог для административного суперпользователя
/boot	Файлы, необходимые для запуска процесса загрузки.
/dev	Содержит специальные файлы устройств, которые используются системой для доступа к оборудованию.

Абсолютные пути и относительные пути

- Файловые системы в ОС Linux чувствительны к регистру.
 - Например: При создании FileCase.txt и filecase.txt в одном и том же каталоге получатся два уникальных файла.
- Существует два пути поиска файла:
 - Абсолютный путь - это полное имя, указывающее точное расположение файлов в иерархии файловой системы. Он начинается с корневого каталога (/) и определяет каждый подкаталог, который необходимо обойти, чтобы получить доступ к определенному файлу.
Например: **/var/log/messages**.
 - Относительный путь – это путь относительно текущего рабочего каталога в командной строке.
Например: **log/messages**.
- Каждый файл в файловой системе имеет уникальное имя абсолютного и/или относительного пути.

Специальные символы и знаки в командной строке

- **Символ тильды (~)**, если ваш текущий рабочий каталог является вашим домашним каталогом. Если он начинается строку символов, отличных от косой черты (/), оболочка интерпретирует строку до этой косой черты как имя пользователя, если оно совпадает, и заменяет строку абсолютным путем к домашнему каталогу этого пользователя. Если имя пользователя не совпадает, вместо него будет использоваться фактическая тильда, за которой следует строка символов.
- Два специальных каталога в верхней части списка относятся к текущему каталогу (.) и родительскому каталогу (..)
- Имена файлов, начинающиеся с точки (.), указывают на скрытые файлы. Вы не можете увидеть их в обычном режиме с помощью `ls` и других команд.
- **Команда `cd ..`** использует .. скрытый каталог для перемещения на один уровень выше к родительскому каталогу без необходимости знать точное родительское имя. Другой скрытый каталог (.) указывает текущий каталог в командах, в которых текущее местоположение является либо исходным, либо целевым аргументом, что позволяет избежать необходимости вводить абсолютный путь к каталогу.

Управление файлами и директориями

Рассмотрим основные команды для управления файлами и директориями

Действия	Синтаксис команды
Где Я? Или путь к текущему каталогу	<code>pwd</code>
Выводит содержимое каталога	<code>ls</code>
Изменить текущий рабочий каталог	<code>cd</code>
Исполнить команду с правами root	<code>sudo</code>
Создание каталогов и файлов	<code>mkdir</code> и <code>touch (echo)</code>
Копирование файла	<code>cp</code>
Перемещение или переименование	<code>mv</code>
Удаление файла	<code>rm</code>

Обзор и примеры основных команд для работы с файлами

- Команда **pwd** отображает полный путь к текущему рабочему каталогу для этой оболочки.
- Команда **ls** выводит содержимое каталога для указанного каталога или, если каталог не указан, для текущего рабочего каталога.

Пример:

- `$ pwd`

`/home/user`

- `$ ls`

Desktop Documents Downloads Music Pictures Public Templates Videos

- Команда **ls** имеет несколько вариантов отображения атрибутов в файлах. Наиболее распространенными и полезными являются **-l** (формат длинного списка), **-a** (все файлы, включая скрытые файлы) и **-R** (рекурсивный, для включения содержимого всех подкаталогов).

Обзор и примеры основных команд для работы с файлами

- Используйте команду `cd`, чтобы изменить текущий рабочий каталог вашей оболочки. Если вы не укажете никаких аргументов для команды, она изменится на ваш домашний каталог.

Пример:

- `$ pwd`

`/home/user`

- `$ cd Videos`

- `$ pwd`

`/home/user/Videos`

- `$ cd /home/user/Documents`

- `$ pwd`

`/home/user/Documents`

- `$ cd`

- `$ pwd`

`/home/user`

Обзор и примеры основных команд для работы с файлами

🔗 Команда **sudo**, (Super User Do).

Позволяет **исполнять команды с правами суперпользователя**. Перед переходом в режим `root` администратора система выполнит запрос пароля и логина `root`.

Пример:

- `$ sudo имя_команды`

Используется в случае, когда не хватает прав для доступа, например, к файлам, созданных другим пользователем.

Команда `mkdir`

Команда `mkdir` создает один или несколько каталогов или подкаталогов. Она принимает в качестве аргументов список путей к каталогам, которые вы хотите создать. Команда `mkdir` завершится с ошибкой, если каталог уже существует, или если вы пытаетесь создать подкаталог в каталоге, который не существует.

Параметр `-p` (parent) создает отсутствующие родительские каталоги для запрошенного места назначения. Используйте команду `mkdir -p` с осторожностью, поскольку орфографические ошибки могут привести к созданию непреднамеренных каталогов без генерации сообщений об ошибках.

Пример:

- `$ mkdir Videos/Watched`
- `$ ls -R Videos`

Videos/:

blockbuster1.ogg blockbuster2.ogg Watched

Команда **touch**

Многие команды и программы могут создавать файлы. Самый простой метод создания файла – это команда **touch**. Она создаст пустой файл, используя указанное имя и местоположение.

В следующем примере команда **touch** создает файлы практики в подкаталогах Documents и Videos

- | | |
|---|--|
| ○ | \$ touch Videos/blockbuster1.ogg |
| ○ | \$ touch Videos/blockbuster2.ogg |
| ○ | \$ touch Documents/thesis_chapter1.odf |
| ○ | \$ touch Documents/thesis_chapter2.odf |

- В приглашении отображается символ тильды (~), если ваш текущий рабочий каталог является вашим домашним каталогом.
- * В ALT Linux для создания файла используется команда **echo**

Обзор и примеры основных команд для работы с файлами

🔄 Команда `cp`

Команда `cp` копирует файл, создавая новый файл, либо в текущем каталоге, либо в указанном каталоге. Она также может копировать несколько файлов в каталог.

Пример:

- `$ cd ../Documents`
- `$ cp thesis_chapter1.odf thesis_chapter2.odf Thesis ProjectX`

`cp: omitting directory `Thesis'`

- `$ ls Thesis ProjectX`

ProjectX:

thesis_chapter1.odf thesis_chapter2.odf

Thesis:

Chapter1 Chapter2 Chapter3

Важно! Если целевой файл уже существует, команда `cp` перезаписывает файл, а каталоги игнорирует.

Обзор и примеры основных команд для работы с файлами

Команда `mv`

Команда `mv` перемещает файлы из одного расположения в другое. Если вы считаете абсолютный путь к файлу его полным именем, перемещение файла фактически равносильно переименованию файла. Содержимое файла остается неизменным.

Пример:

- `$ cd ../Documents`

- `$ ls -l thesis*`

```
-rw-rw-r--. 1 user user 0 Feb 6 21:16 thesis_chapter1.odf
```

```
-rw-rw-r--. 1 user user 0 Feb 6 21:16 thesis_chapter2.odf
```

- `$ mv thesis_chapter2.odf thesis_chapter2_reviewed.odf`

- `$ ls -l thesis*`

```
-rw-rw-r--. 1 user user 0 Feb 6 21:16 thesis_chapter1.odf
```

```
-rw-rw-r--. 1 user user 0 Feb 6 21:16 thesis_chapter2_reviewed.odf
```

Обзор и примеры основных команд для работы с файлами

Команда `rm`

Команда `rm` удаляет файлы. По умолчанию `rm` не будет удалять каталоги, содержащие файлы, если вы не добавите опцию `-r` или `--recursive`.

Пример:

- `$ ls -l thesis*`

```
-rw-rw-r--. 1 user user 0 Feb 6 21:16 thesis_chapter2_reviewed.odf
```

- `$ rm thesis_chapter2_reviewed.odf`

- `$ ls -l thesis*`

`ls: cannot access 'thesis*': No such file or directory`

Важно! Нет ни функции восстановления из командной строки, ни "корзины", из которой вы можете восстановить файлы, подготовленные к удалению.

Восстановление возможно только специальными средствами!

Создание ссылок между файлами

🔗 Жесткие ссылки:

Вы можете узнать, имеет ли файл несколько жестких ссылок, с помощью команды `ls -l`. Например:

- `$ pwd`

`/home/user`

- `$ ls -l newfile.txt`

`-rw-r--r--. 1 user user 0 Mar 11 19:19 newfile.txt`

В этом примере один абсолютный путь: `/home/user/newfile.txt`

Команду `ln` служит для создания новой жесткой ссылки (другое имя), которая указывает на существующий файл. Команде нужны по крайней мере два аргумента:

- путь к существующему файлу и
- путь к жесткой ссылке, которую вы хотите создать.

Создание ссылок между файлами

В следующем примере создается жесткая ссылка с именем newfile-link2.txt для существующего файла newfile.txt в каталоге /tmp.

- `$ ln newfile.txt /tmp/newfile-hlink2.txt`
- `$ ls -l newfile.txt /tmp/newfile-hlink2.txt`

```
-rw-rw-r--. 2 user user 12 Mar 11 19:19 newfile.txt
```

```
-rw-rw-r--. 2 user user 12 Mar 11 19:19 /tmp/newfile-hlink2.txt
```

Если вы хотите узнать, являются ли два файла жесткими ссылками друг на друга, один из способов - использовать опцию `-i` с командой `ls` для отображения номера индекса файлов.

- `$ ls -il newfile.txt /tmp/newfile-hlink2.txt`

```
8924107 -rw-rw-r--. 2 user user 12 Mar 11 19:19 newfile.txt
```

```
8924107 -rw-rw-r--. 2 user user 12 Mar 11 19:19 /tmp/newfile-hlink2.txt
```

- Жесткие ссылки имеют некоторые ограничения:
 - Жесткие ссылки можно использовать только с обычными файлами.
 - Жесткие ссылки можно использовать только в том случае, если оба файла находятся в одной файловой системе.

Вы можете использовать команду **df** для составления списка каталогов, находящихся в разных файловых системах. Например, вы можете увидеть вывод, подобный следующему:

- `$ df`

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
devtmpfs	886788	0	886788	0%	/dev
tmpfs	902108	0	902108	0%	/dev/shm
tmpfs	902108	8696	893412	1%	/run
tmpfs	902108	0	902108	0%	/sys/fs/cgroup
/dev/mapper/rhel_rhel8--root	10258432	1630460	8627972	16%	/
/dev/sda1	1038336	167128	871208	17%	/boot
tmpfs	180420	0	180420	0%	/run/user/1000

Создание ссылок между файлами

🔗 Символические (мягкие) ссылки:

Команда `ln -s` создает символическую ссылку.

Символическая ссылка - это не обычный файл, а файл особого типа, указывающий на существующий файл или каталог.

Символические ссылки имеют некоторые преимущества перед жесткими:

- Они могут связывать два файла в разных файловых системах.
- Они могут указывать на каталог или специальный файл, а не только на обычный файл.

В следующем примере команда `ln -s` используется для создания новой символической ссылки для существующего файла `/home/user/newfile-link2.txt` с именем `/tmp/newfile-symlink.txt`.

- `$ ln -s /home/user/newfile-link2.txt /tmp/newfile-symlink.txt`
- `$ ls -l newfile-link2.txt /tmp/newfile-symlink.txt`

```
-rw-rw-r--. 1 user user 12 Mar 11 19:19 newfile-link2.txt
```

```
lrwxrwxrwx. 1 user user 11 Mar 11 20:59 /tmp/newfile-symlink.txt -> /home/user/newfile-link2.txt
```

- `$ cat /tmp/newfile-symlink.txt`

Soft Hello World

Создание ссылок между файлами

Когда исходный обычный файл будет удален, символическая ссылка по-прежнему будет указывать на файл, но цель исчезнет. Символическая ссылка, указывающая на отсутствующий файл, называется «зависшей символической ссылкой (dangling soft link)».

- `$ rm -f newfile-link2.txt`

- `$ ls -l /tmp/newfile-symlink.txt`

```
lrwxrwxrwx. 1 user user 11 Mar 11 20:59 /tmp/newfile-symlink.txt -> /home/user/  
newfile-link2.txt
```

- `$ cat /tmp/newfile-symlink.txt`

```
cat: /tmp/newfile-symlink.txt: No such file or directory
```

Создание ссылок между файлами

Символическая ссылка может указывать на каталог. В этом случае символическая ссылка действует, как каталог. Переход по символической ссылке с помощью команды `cd` сделает текущий рабочий каталог связанным каталогом. Некоторые инструменты могут отслеживать тот факт, что вы перешли по символической ссылке, чтобы попасть туда. Например, по умолчанию `cd` обновит ваш текущий рабочий каталог, используя имя символической ссылки, а не имя фактического каталога (есть опция `-P`, которая вместо этого обновит его именем фактического каталога.)

В следующем примере создается программная ссылка с именем `/home/user/configfiles`, указывающая на каталог `/etc`.

- `$ ln -s /etc /home/user/configfiles`
- `$ cd /home/user/configfiles`
- `$ pwd`

`/home/user/configfiles`

Использование шаблонов и переменных в командной строке

- В русскоязычной литературе «**expansion**» из документации **Bash** часто переводят как «подстановка». Внешне этот процесс выглядит именно так – вместо одних символов в командной строке появляются другие символы или новые слова. Но если заглянуть глубже, то имеет место не механическая подстановка («найти символ звёздочка и заменить её на набор слов bin Видео Документы Загрузки Изображения Музыка Общедоступные Рабочий стол Шаблоны»), а именно расширение, раскрытие значения. Значение, которое может иметь * в командной строке, зависит от того, в какой директории пользователь запустил эту команду, и что содержится в этой директории.
- **Универсализация (Globbing)** - это операция синтаксического анализа команд оболочки, которая расширяет возможности работы в командной строке при помощи шаблона подстановки в списке совпадающих имен путей. Перед выполнением команды метасимволы командной строки заменяются списком совпадений. Шаблоны, которые не возвращают совпадений, отображают исходный запрос шаблона как буквальный текст.

Ниже приведены общие метасимволы и классы шаблонов.

Использование шаблонов и переменных в командной строке

Подстановочные символы	Значение
*	Любая строка из нуля или более символов.
?	Любой одиночный символ.
[abc ...]	Любой символ в закрытом классе (в квадратных скобках).
[! abc ...]	Любой один символ, не входящий в закрытый класс.
[^ abc ...]	Любой один символ, не входящий в закрытый класс.
[[:alpha:]]	Любой буквенный символ.
[[:lower:]]	Любой символ нижнего регистра.
[[:upper:]]	Любой символ верхнего регистра.
[[:alnum:]]	Любой буквенный символ или цифра.
[[:punct:]]	Любой печатный символ, кроме пробела или буквенно-цифрового
[[:digit:]]	Любая отдельная цифра от 0 до 9.
[[:space:]]	Любой одиночный символ пробела. (табуляция, перевод строки)

Использование шаблонов и переменных в командной строке

Для следующих нескольких примеров представьте, что вы выполнили следующие команды, чтобы создать несколько файлов для примеров:

```
○ $ mkdir glob; cd glob
```

```
○ $ touch alfa bravo charlie delta echo able baker cast dog easy
```

```
○ $ ls
```

```
able alfa baker bravo cast charlie delta dog easy echo
```

В первом примере будет использоваться простое сопоставление шаблонов с символами звездочки (*) и вопросительного знака (?), а также класс символов для сопоставления некоторых из этих имен файлов.

```
○ $ ls a*
```

```
able alfa
```

```
○ $ ls *a*
```

```
able alfa baker bravo cast charlie delta easy
```

```
○ $ ls [ac]*
```

```
able alfa cast charlie
```

Использование шаблонов и переменных в командной строке

В следующем примере команда **echo** используется для отображения значения символа тильды. Команду **echo** также можно использовать для отображения значений скобок, символов раскрытия переменных и других.

```
○ $ ls ~root
```

```
/root
```

```
○ $ ls ~user
```

```
/home/user
```

```
○ $ ls ~/glob
```

```
able alfa baker bravo cast charlie delta dog easy echo
```

```
○ $ echo ~/glob
```

```
/home/user/glob
```

Использование шаблонов и переменных в командной строке

Расширение скобки используется для создания произвольных строк символов. Фигурные скобки содержат список строк, разделенных запятыми, или выражение последовательности. Результат включает текст, предшествующий или следующий за определением скобки. Пары скобок могут быть вложены одна в другую. Также синтаксис с двумя точками (..) расширяется до последовательности, так что {m..p} расширяется до m n o p.

```
○ $ echo {Sunday,Monday,Tuesday,Wednesday}.log
```

```
Sunday.log Monday.log Tuesday.log Wednesday.log
```

```
○ $ echo file{1..3}.txt
```

```
file1.txt file2.txt file3.txt
```

```
○ $ echo file{a..c}.txt
```

```
filea.txt fileb.txt filec.txt
```

```
○ $ echo file{a,b}{1,2}.txt
```

```
filea1.txt filea2.txt fileb1.txt fileb2.txt
```

```
○ $ echo file{a{1,2},b,c}.txt
```

```
filea1.txt filea2.txt fileb.txt filec.txt
```

Использование шаблонов и переменных в командной строке

Переменная действует как именованный контейнер, который может хранить значение в памяти. Переменные упрощают доступ к сохраненным данным, а также их изменение из командной строки или в сценарии оболочки.

Вы можете присвоить данные как значение переменной, используя следующий синтаксис:

- `$ VARIABLENAME=value`

Вы можете использовать расширение переменной, чтобы преобразовать имя переменной в ее значение в командной строке. Если строка начинается со знака доллара (\$), оболочка попытается использовать оставшуюся часть этой строки в качестве имени переменной и заменить ее значением переменной.

- `$ USERNAME=operator`

- `$ echo $USERNAME`

Чтобы избежать ошибок из-за других расширений оболочки, вы можете заключить имя переменной в фигурные скобки, например {VARIABLENAME}.

- `$ USERNAME=operator`

- `echo ${USERNAME}`

operator

Использование шаблонов и переменных в командной строке

Подстановка команд позволяет выводом команды заменить саму команду в командной строке. Подстановка команд происходит, когда команда заключена в круглые скобки и ей предшествует знак доллара (\$). Форма **\$(command)** может содержать несколько расширений команд друг в друге.

- `$ echo Today is $(date +%A).`

Today is Wednesday.

- `$ echo The time is $(date +%M) minutes past $(date +%l%p).`

The time is 26 minutes past 11AM.

Защита аргументов от расширения (выполнение подмены)

Обратная косая черта (\) - это символ экранирования в оболочке Bash. Он защищает символы, следующего сразу за ним, от выполнения расширения.

- `$ echo The value of $HOME is your home directory.`

The value of /home/user is your home directory.

- `$ echo The value of \ $HOME is your home directory.`

The value of \$HOME is your home directory.

Использование шаблонов и переменных в командной строке

Используйте двойные кавычки, чтобы запретить подстановку символов и раскрытие оболочки, но все же разрешите подстановку команд и переменных.

```
○ $ myhost=$(hostname -s); echo $myhost
```

host

```
○ $ echo "***** hostname is ${myhost} *****"
```

```
***** hostnameishost *****
```

Используйте одинарные кавычки для буквального толкования всего текста.

```
○ $ echo "Will variable $myhost evaluate to $(hostname -s)? "
```

Will variable myhost evaluate to host?

```
○ $ l variable $myhost evaluate to $(hostname -s)?'
```

Will variable \$myhost evaluate to \$(hostname -s)?

Следующая тема

- 🔄 Управление хранением
- 🔄 Файловые системы и точки монтирование
 - Файловые системы, устройства хранения и блочные устройства
 - Разделы дисков и логические тома
 - Монтирование и размонтирование файловых систем
- 🔄 Управление файлами
 - Иерархия файловой системы Linux
 - Имена файлов и пути к ним
 - Управление файлами и директориями
 - Создание ссылок между файлами
 - Использование шаблонов и переменных в командной строке
- 🔄 **Поиск файлов в файловых системах**

Поиск файлов в файловых системах.

Поиск файлов.

- Системному администратору нужны инструменты для поиска файлов, соответствующих определенным критериям, в файловой системе.
- Команда **locate** ищет в предварительно сгенерированном индексе имена файлов или пути к файлам и мгновенно возвращает результаты.
- Команда **find** выполняет поиск файлов в реальном времени путём обхода иерархии файловой системы.

Поиск файлов по имени.

База данных местоположения автоматически обновляется каждый день. Однако в любой момент пользователь **root** может выполнить команду **updatedb**, чтобы выполнить немедленное обновление.

```
root@host:~# updatedb
```


Поиск файлов в файловых системах.

Поиск файлов.

- Поиск файлов с **"passwd"** в имени или пути в деревьях каталогов, доступных для чтения пользователем на хосте

```
user@host:~$ locate passwd
/etc/passwd
/etc/passwd-
/etc/pam.d/passwd
/etc/security/opasswd
/usr/bin/gpasswd
/usr/bin/grub2-mkpasswd-pbkdf2
/usr/bin/lppasswd
/usr/bin/passwd
...output omitted...
```

Поиск файлов в файловых системах.

Поиск файлов.

- Результаты возвращаются, даже если имя файла или путь лишь частично совпадают с поисковым запросом.

```
root@host:~# locate image
/etc/selinux/targeted/contexts/virtual_image_context
/usr/bin/grub2-mkimage
/usr/lib/sysimage
/usr/lib/dracut/dracut.conf.d/02-generic-image.conf
/usr/lib/firewalld/services/ovirt-imageio.xml
/usr/lib/grub/i386-pc/lnxboot.image
...output omitted...
```

Поиск файлов в файловых системах.

Поиск файлов.

- Параметр **-i** выполняет поиск без учёта регистра. С этой опцией все возможные комбинации прописных и строчных букв соответствуют поиску:

```
user@host:~$ locate -i messages
...output omitted...
/usr/share/vim/vim80/lang/zh_TW/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW/LC_MESSAGES/vim.mo
/usr/share/vim/vim80/lang/zh_TW.UTF-8/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW.UTF-8/LC_MESSAGES/vim.mo
/usr/share/vim/vim80/syntax/messages.vim
/usr/share/vim/vim80/syntax/msmessages.vim
/var/log/messages
```

Поиск файлов.

- Параметр **-n** ограничивает количество результатов поиска, возвращаемых командой **locate**. В следующем примере результаты поиска, возвращаемые командой **locate**, ограничиваются первыми пятью совпадениями:

```
user@host:~$ locate -n 5 snow.png
/usr/share/icons/HighContrast/16x16/status/weather-snow.png
/usr/share/icons/HighContrast/22x22/status/weather-snow.png
/usr/share/icons/HighContrast/24x24/status/weather-snow.png
/usr/share/icons/HighContrast/256x256/status/weather-snow.png
/usr/share/icons/HighContrast/32x32/status/weather-snow.png
```

Поиск файлов в файловых системах.

Поиск файлов в реальном времени.

- Команда **find** находит файлы, выполняя поиск в реальном времени в иерархии файловой системы.

Например, чтобы найти файлы с именем **sshd_config**, начиная с каталога **/**, выполните следующую команду:

```
root@host:~# find / -name sshd_config
/etc/ssh/sshd_config
```

В следующем примере выполняется поиск файлов, заканчивающихся на **.txt**, начиная с каталога **/**:

```
root@host:~# find / -name '*.txt'
/etc/pki/nssdb/pkcs11.txt
/etc/brltty/brl-lt-all.txt
/etc/brltty/brl-mb-all.txt
/etc/brltty/brl-md-all.txt
/etc/brltty/brl-mn-all.txt
...output omitted...
```

Поиск файлов в файловых системах.

Поиск файлов в реальном времени.

Чтобы найти файлы в каталоге `/etc/`, содержащие слово **"pass"** в любом месте своего имени, на хосте, выполните следующую команду:

```
root@host:~# find /etc -name '*pass*'
/etc/security/opasswd
/etc/pam.d/passwd
/etc/pam.d/password-auth
/etc/passwd-
/etc/passwd
/etc/authselect/password-auth
```

Чтобы выполнить поиск по заданному имени файла без учета регистра, используйте параметр **-iname**, за которым следует имя файла для поиска. Для поиска файлов, содержащих слово **"messages"** в своём имени без учёта регистра, в каталоге `/` на хосте, выполните следующую команду:

```
root@host:~# find / -iname '*messages*'
...output omitted...
/usr/share/vim/vim80/lang/zh_CN.UTF-8/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_CN.cp936/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW.UTF-8/LC_MESSAGES
/usr/share/vim/vim80/syntax/messages.vim
/usr/share/vim/vim80/syntax/msmessages.vim
```

Поиск файлов в файловых системах.

Поиск файлов в зависимости от владельца файла или разрешений для файла

Команда **find** может искать файлы в зависимости от их владельца или разрешений. При поиске по владельцу полезными параметрами являются **-user**, **-group**, которые ищут по имени, и **-uid**, **-gid**, которые ищут по **ID**.

Найдите файлы, принадлежащие пользователю **user**, в каталоге **/home/user** на хосте.

```
user@host:~$ find -user user
.
./bash_logout
./bash_profile
./bashrc
./bash_history
```

Найдите файлы, принадлежащие пользователю группы **user**, в каталоге **/home/user** на хосте.

```
user@host:~$ find -group user
.
./bash_logout
./bash_profile
./bashrc
./bash_history
```

Поиск файлов в файловых системах.

Поиск файлов в зависимости от владельца файла или разрешений для файла

Найдите файлы, принадлежащие пользователю с **ID 1000**, в каталоге **/home/user** на хосте.

```
user@host:~$ find -uid 1000
.
./bash_logout
./bash_profile
./bashrc
./bash_history
```

Найдите файлы, принадлежащие группе с идентификатором **1000**, в каталоге **/home/user** на хосте.

```
user@host:~$ find -gid 1000
.
./bash_logout
./bash_profile
./bashrc
./bash_history
```


Поиск файлов в файловых системах.

🔍 Поиск файлов в зависимости от владельца файла или разрешений для файла

Следующая команда соответствует любому файлу, для которого пользователь имеет разрешения на чтение, запись и выполнение, члены группы имеют разрешения на чтение и запись, а другие имеют доступ только для чтения:

```
root@host:~# find /home -perm 764
```

Чтобы сопоставить файлы, для которых у пользователя есть как минимум права на запись и выполнение, а у группы есть как минимум права на запись, а у других есть как минимум права на чтение:

```
root@host:~# find /home -perm -324
```

Чтобы сопоставить файлы, для которых у пользователя есть разрешения на чтение, или у группы есть хотя бы разрешения на чтение, или у других есть хотя бы доступ на запись:

```
root@host:~# find /home -perm /442
```

При использовании с "/" или "-" значение "0" работает как подстановочный знак, поскольку означает разрешение как минимум ничего (*permission of at least nothing*). Чтобы сопоставить любой файл в каталоге /home/user, к которому у других есть хотя бы доступ для чтения на хосте, выполните:

```
user@host:~$ find -perm -004
```

Найдите все файлы в каталоге /home/user, где у прочих (other) есть разрешения на запись (write) на хосте.

```
user@host:~$ find -perm -002
```

Поиск файлов по размеру

Команда **find** может искать файлы, размер которых соответствует размеру, указанному в параметре **-size**, за которым следует числовое значение и единица измерения. Используйте следующий список в качестве единиц с параметром **-size**:

- ✓ **k** – для килобайта
- ✓ **M** – для мегабайта
- ✓ **G** – для гигабайта

В приведенном ниже примере показано, как искать файлы размером 10 мегабайт с округлением в большую сторону.

```
user@host:~$ find -size 10M
```

Для поиска файлов размером более 10 гигабайт выполните команду:

```
user@host:~$ find -size +10G
```

Вывести список всех файлов размером менее 10 килобайт.

```
user@host:~$ find -size -10k
```

Поиск файлов в файловых системах.

Поиск файлов по времени модификации

Параметр `-mmin`

Например, чтобы найти все файлы, содержимое которых было изменено **120 минут назад** на хосте, выполните команду:

```
root@host:~# find / -mmin 120
```

Модификатор `"+"` перед количеством минут позволяет искать все файлы в каталоге `/`, которые были изменены **более n** минут назад. В этом примере перечислены файлы, которые были изменены **более 200 минут назад**.

```
root@host:~# find / -mmin +200
```

Модификатор `"-"` меняет поиск так, что можно искать все файлы в каталоге `/`, которые были изменены **менее n** минут назад. В этом примере перечислены файлы, которые были изменены **менее 150 минут назад**.

```
root@host:~# find / -mmin -150
```

Поиск файлов в файловых системах.

Поиск файлов по их типу

Параметр **-type** в команде **find** ограничивает область поиска заданным типом файла. Используйте следующий список, чтобы передать необходимые флаги, чтобы ограничить область поиска:

- ✓ **f** – для обычного файла
- ✓ **d** – для каталога
- ✓ **l** – для символической-ссылки
- ✓ **b** – для блочного устройства

Найдите все символические ссылки на хосте.

```
# find / -type l
```

Сгенерируйте список всех блочных устройств в каталоге **/dev** на хосте:

```
# find /dev -type b
```

```
/dev/vda1  
/dev/vda
```

Искать все обычные файлы с более чем одной жесткой ссылкой на хосте:

```
# find / -type f -links +1
```

В данной лекции были рассмотрены следующие темы:

- 🌀 Управление хранением

- 🌀 Файловые системы и точки монтирование

- Файловые системы, устройства хранения и блочные устройства
- Разделы дисков и логические тома
- Монтирование и размонтирование файловых систем

- 🌀 Управление файлами

- Иерархия файловой системы Linux
- Имена файлов и пути к ним
- Управление файлами и директориями
- Создание ссылок между файлами
- Использование шаблонов и переменных в командной строке

- 🌀 Поиск файлов в файловых системах